



Da MCP tools a Distributed Intelligence

Come DDD, MCP, A2A, RAG, Wiki e observability
cambiano il modo in cui progettiamo sistemi AI-native



La tesi

**La vera sfida non è collegare un LLM a qualche API.
È progettare come l'intelligenza fluisce nell'organizzazione.**

BrewUp ERP come laboratorio

Un ERP demo abbastanza piccolo da essere spiegato, ma abbastanza realistico da far emergere i problemi.

MasterData

catalogo, clienti
identità del dominio

Sales

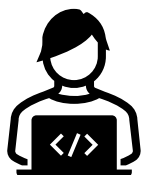
ordini, domanda
segnali commerciali

Warehouse

stock, soglie
impatto operativo

Knowledge

documenti, policy
conoscenza esplicita



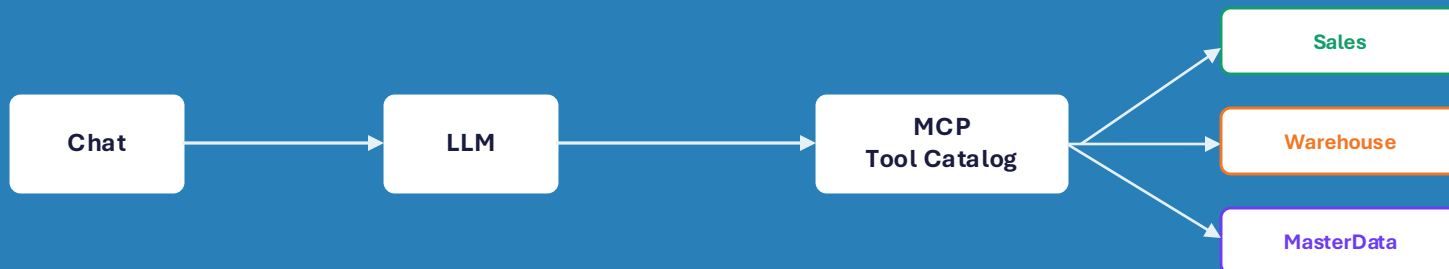
Vediamo repo,
branch `DDD`

Creare un agente è facile. Progettarlo no.



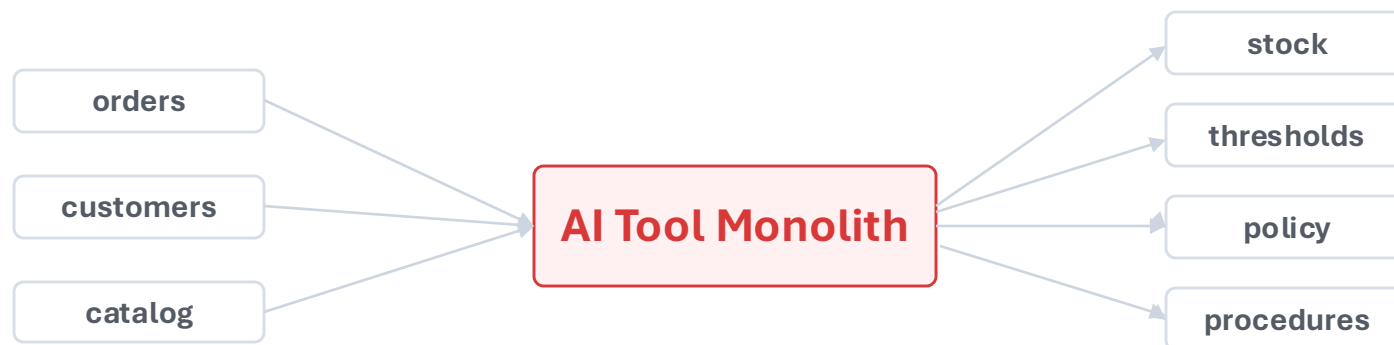
1 Esperimento: dare all'AI accesso all'ERP

MCP come superficie scopribile delle capability applicative.



Problema: il Tool Catalog diventa un monolite

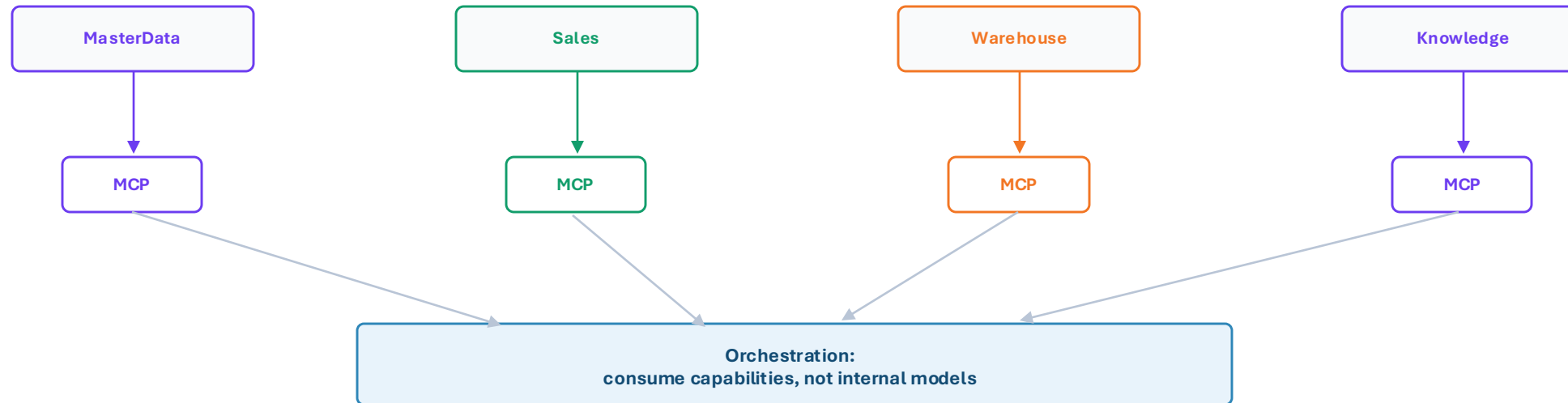
Una superficie unica tende ad accumulare linguaggi, regole e responsabilità diverse.



Se un unico agente conosce tutto, abbiamo solo cambiato interfaccia al monolite.

Decisione: un MCP Server per Bounded Context

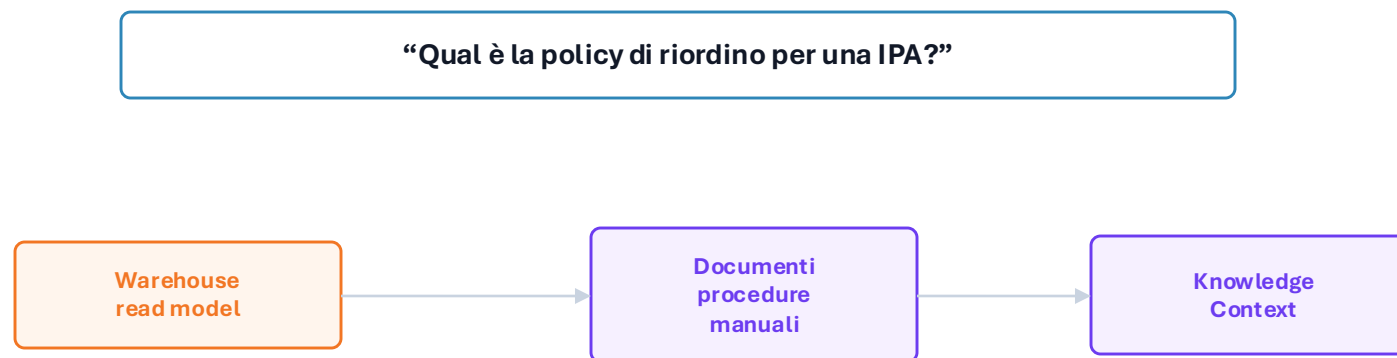
Non soprattutto per deployment. Per ownership del linguaggio e della conoscenza.



**Bounded Contexts don't just separate code.
They separate knowledge.**

Il dominio non vive tutto nel database

Lo stato operativo risponde a una parte delle domande. Le policy spesso vivono altrove.

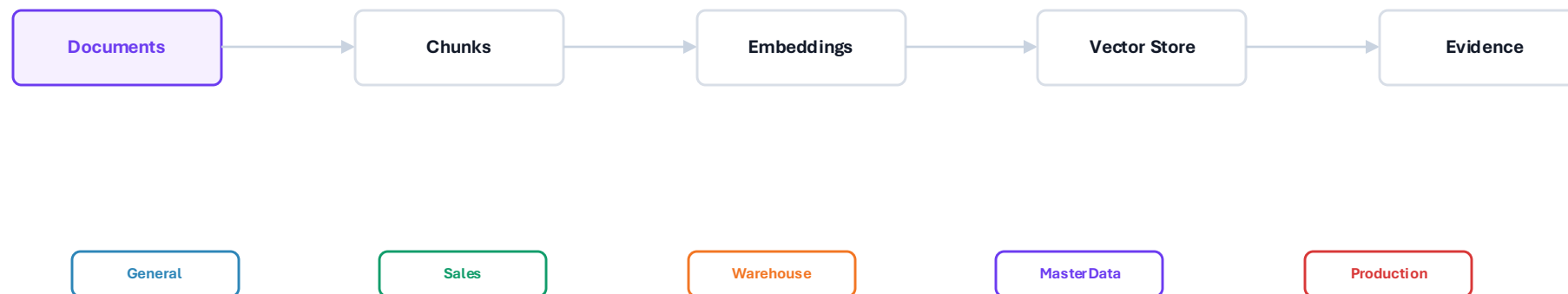


Ci sono domande che nessun MCP operativo può risolvere da solo: serve ulteriore conoscenza.

Dove sta questa conoscenza?

Knowledge Context: RAG con confini di dominio

La documentazione diventa interrogabile senza diventare verità operativa.



Anche la conoscenza documentale ha uno scope.

4 Retrieval non significa conoscenza

Il RAG trova frammenti rilevanti. Il sistema può costruire una memoria più stabile?



La Wiki non sostituisce le fonti.
Le organizza in una conoscenza derivata e verificabile.

LLM Wiki: conoscenza derivata con provenance

La parte importante non è “generare pagine”, ma conservare il legame tra claim e fonti.

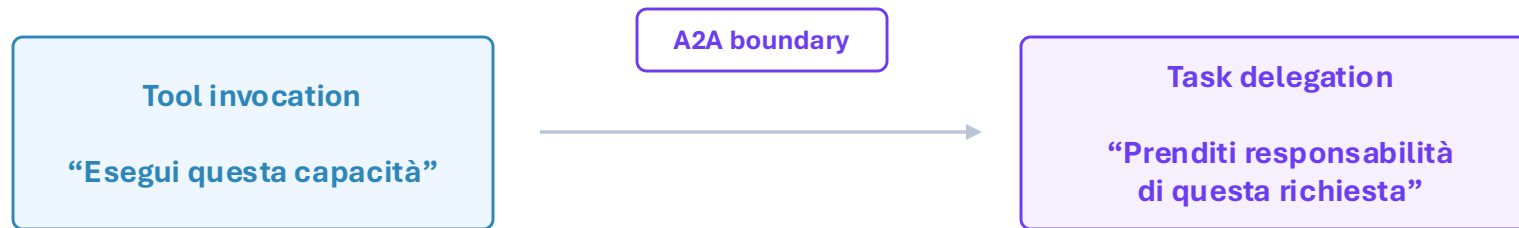


```
query_wiki  
get_wiki_page  
get_wiki_page_evidence
```

Derived knowledge is not operational truth.
Stock, ordini e disponibilità restano nei rispettivi Bounded Context.

Un tool non è un agente

Quando una capability ha responsabilità, discovery, error handling e autonomia evolutiva, non stiamo solo invocando una funzione.

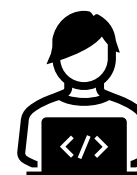


MCP: What can you do?

A2A: Can you take responsibility for this task?

Knowledge Agent: dal tool al collaboratore

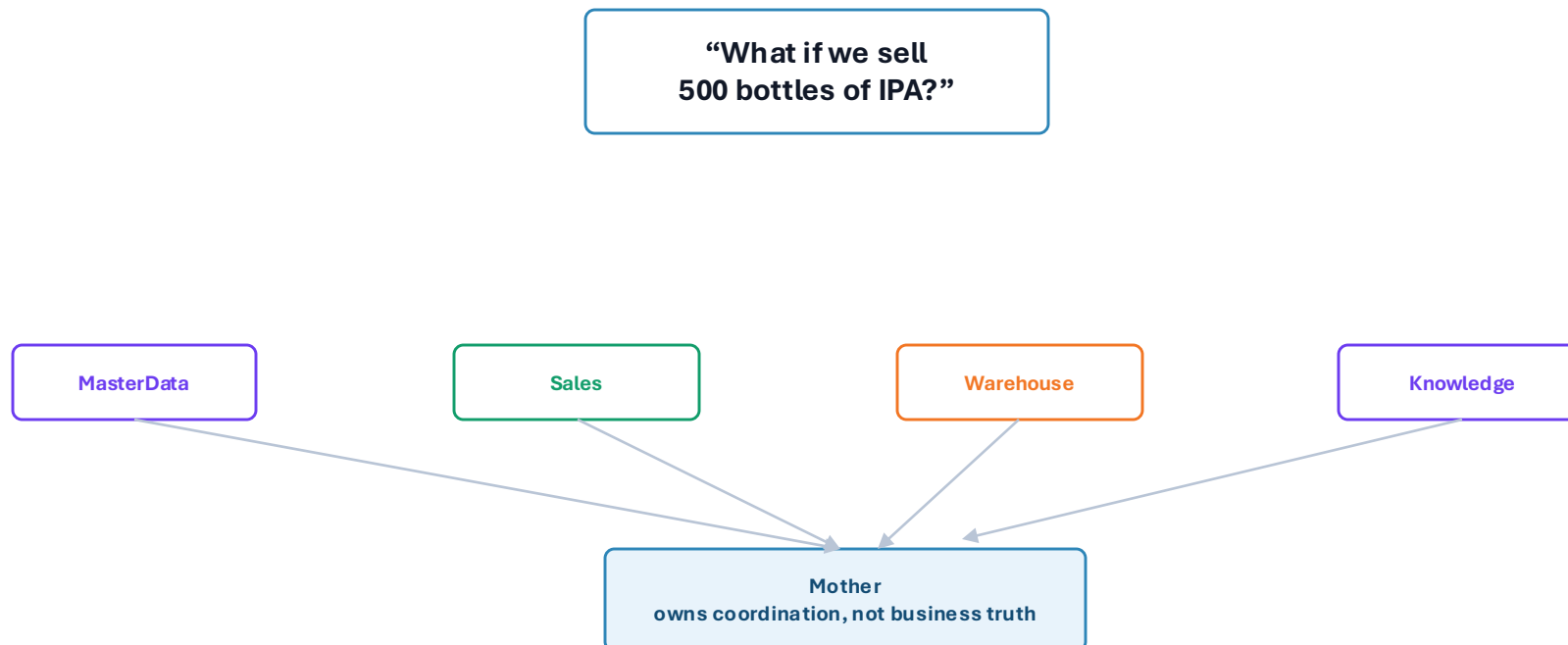
Mother delega una domanda di conoscenza. L'agente scopre e usa i tool MCP del Knowledge Context.



Vediamo il codice,
branch `a2a-no-
framework`???

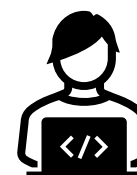
Chi coordina tutto questo?

Alcune domande non appartengono a nessun singolo Bounded Context.



What-if workflow: collaborazione tra contesti

Mother non inventa fatti: pianifica, delega, correla e valuta.



Vediamo il codice,
branch ``mother-with-mcp-and-rag``

**Bounded Contexts own truth.
Mother owns coordination.**

“Ha funzionato” non è abbastanza

In un sistema agentico dobbiamo osservare sia l'infrastruttura sia il ragionamento.

Technical observability

What happened?

HTTP · SQL · MCP · model call · latency · errors

Semantic observability

Why did it happen?

AgentRun · workflow · capability · handoff · outcome

**Non osserviamo solo servizi.
Osserviamo decisioni.**

Evaluation: abbiamo abbastanza evidenza?

La domanda non è solo “quali agenti hanno risposto?”, ma “il workflow è completo abbastanza da fidarci?”.



Prodotto risolto



Sales evidence



Warehouse evidence



Knowledge evidence

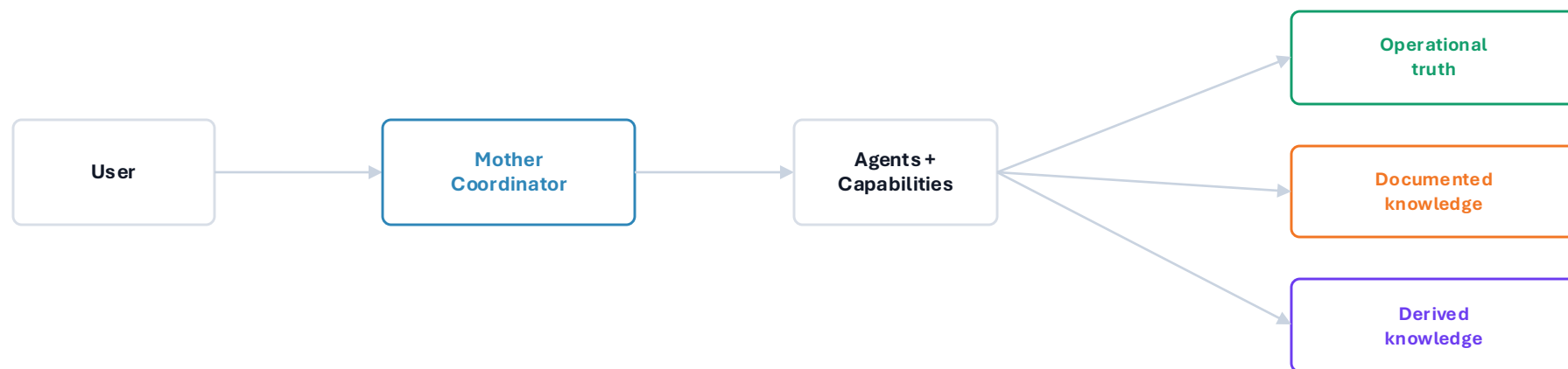


Outcome

Observability tells us what the system did.
Evaluation tells us whether what it did was enough.

Il sistema finale: intelligence flow

Non un agente centrale che sa tutto, ma un flusso coordinato tra contesti, agenti e conoscenza.



Ok. Enough diagrams.
Let's see what we actually built.

Show me the system

Un tour mirato: solo i punti in cui le decisioni architetturali diventano codice.

1

Bounded Contexts

Sales · Warehouse
MasterData · Knowledge
con MCP Server separati

2

KnowledgeTools

search_knowledge_base
query_wiki
get_wiki_page
get_wiki_page_evidence

3

Knowledge Agent

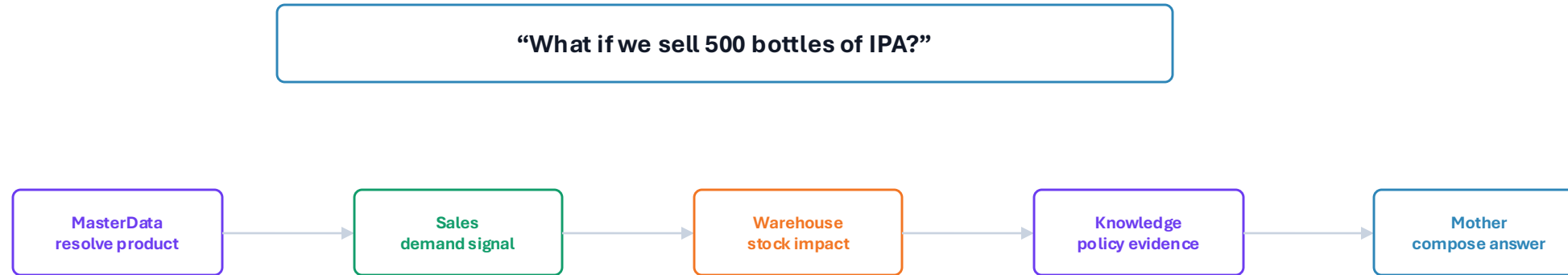
Mother → A2A → Agent → MCP →
Knowledge

4

MotherCoordinator

MasterData → Sales →
Warehouse
→ Knowledge → Evaluation

Demo end-to-end: una sola domanda



La demo non mostra tanti casi. Mostra un caso che attraversa tutta l'architettura.

Apriamo il trace

La stessa richiesta diventa leggibile come workflow semantico.

```
HTTP POST /chat
└─ AgentRun
   └─ invoke_workflow brewup.what-if
      ├── invoke_agent MasterDataAgent
      ├── invoke_agent SalesAgent
      ├── invoke_agent WarehouseAgent
      ├── invoke_agent KnowledgeAgent
      └─ evaluation
```

**Prima vediamo la risposta.
Poi vediamo come è stata costruita.**

Questo è il ponte tra observability tecnica, observability semantica ed evaluation.

Da tool invocation a intelligence flow



La domanda non è più: “quali tool posso chiamare?”
La domanda diventa: “come fluisce la conoscenza?”

Thanks!



 aacerbis
 alberto.acerbis@intre.it



 ferdinandasantacroce
 ferdinando.santacroce@gmail.com



<https://github.com/BrewUp/BrewUpErp>